LeetCode HOT 100

PART 1

Hashing

哈希

1. 两数之和 (No.1) EASY

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** *`target`* 的那 **两个** 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

示例 1：

> **输入：** nums = [2,7,11,15], target = 9
> **输出：** [0,1]
> **解释：** 因为 nums[0] + nums[1] == 9 ，返回 [0, 1] 。

示例 2：

> **输入：** nums = [3,2,4], target = 6
> **输出：** [1,2]

示例 3：

> **输入：** nums = [3,3], target = 6
> **输出：** [0,1]

提示：

- $2 <= nums.length <= 10^4$
- $-10^9 <= nums[i] <= 10^9$
- $-10^9 <= target <= 10^9$
- **只会存在一个有效答案**

进阶： 你可以想出一个时间复杂度小于 $O(n^2)$ 的算法吗？

1. 两数之和 (No.1) EASY

思路

创建一个哈希表，对于每一个num，key=target-num，value=下标，当某个num匹配到哈希表的key就找到了组合。

```
class Solution(object):
    def twoSum(self, nums, target):
        """
        :type nums: List[int]
        :type target: int
        :rtype: List[int]
        """
        num_map = {}  # key是每个数与target的差值，value是下标
        for i in range(len(nums)):
            if nums[i] in num_map:
                return [i, num_map[nums[i]]]
            num_map[target - nums[i]] = i
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(n)$

2. 字母异位词分组 (No.49) MEDIUM

给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。

示例 1:

> **输入:** strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
>
> **输出:** [["bat"],["nat","tan"],["ate","eat","tea"]]
>
> **解释:**
>
> - 在 strs 中没有字符串可以通过重新排列来形成 `"bat"`。
> - 字符串 `"nat"` 和 `"tan"` 是字母异位词，因为它们可以重新排列以形成彼此。
> - 字符串 `"ate"` ， `"eat"` 和 `"tea"` 是字母异位词，因为它们可以重新排列以形成彼此。

示例 2:

> **输入:** strs = [""]
>
> **输出:** [[""]]

示例 3:

> **输入:** strs = ["a"]
>
> **输出:** [["a"]]

提示:

- `1 <= strs.length <= 10^4`
- `0 <= strs[i].length <= 100`
- `strs[i]` 仅包含小写字母

2. 字母异位词分组 (No.49) MEDIUM

思路

创建一个哈希表，key=每组异位词排序后的字符串，value=异位词列表，python中注意列表不能作为key。

```
class Solution(object):
    def groupAnagrams(self, strs):
        """
        :type strs: List[str]
        :rtype: List[List[str]]
        """
        alpha_map = defaultdict(list)  # key是排序后字符串，value是异位词列表
        for string in strs:
            sorted_str = "".join(sorted(string))  # sorted返回排序后的单字符列表
            alpha_map[sorted_str].append(string)
        return [x for x in alpha_map.values()]
```

时间复杂度：$\mathcal{O}(n \cdot k\, logk)$， n为字符串数量， k为最大字符串长度；**空间复杂度**：$\mathcal{O}(nk)$

3. 最长连续序列 (No.128) MEDIUM

给定一个未排序的整数数组 `nums` ，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 `O(n)` 的算法解决此问题。

示例 1：

> **输入：** nums = [100,4,200,1,3,2]
> **输出：** 4
> **解释：** 最长数字连续序列是 [1, 2, 3, 4]。它的长度为 4。

示例 2：

> **输入：** nums = [0,3,7,2,5,8,4,6,0,1]
> **输出：** 9

示例 3：

> **输入：** nums = [1,0,1,2]
> **输出：** 3

提示：

- $0 <= nums.length <= 10^5$
- $-10^9 <= nums[i] <= 10^9$

3. 最长连续序列 (No.128) MEDIUM

思路

创建一个哈希表，key=每个数字。遍历哈希表，如果这个数字-1不在哈希表中，说明这个数字是某连续序列开头的数字，因此从这个数字开始算长度更新结果res；反之这个数字是连续序列中间的数字，就不必算长度。

```
class Solution(object):
    def longestConsecutive(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        num_map = set(nums)  # 不需要value，用set存储
        res = 0
        for num in num_map:
            if num - 1 not in num_map:  # 仅当num-1不在哈希表中需要从头开始计数
                ans, tmp = 0, num
                while tmp in num_map:
                    ans += 1
                    tmp += 1
                res = max(res, ans)
        return res
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(n)$